

Polymorphism

Polymorphism

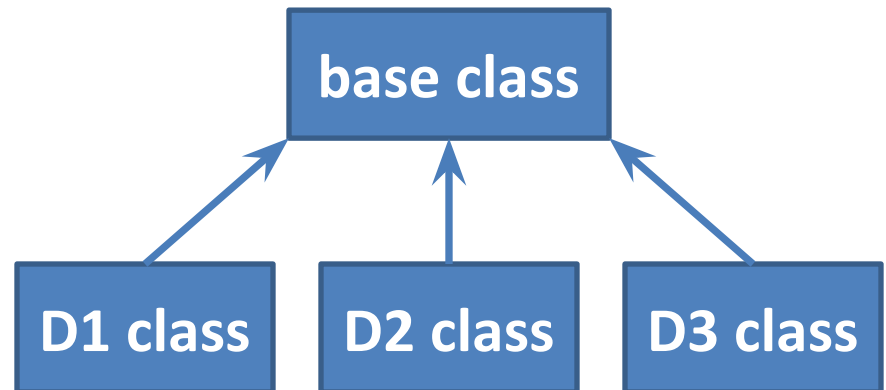
- Combination of two Greek words
- Poly (**many**) morphism (**form**)
 - Water -> Solid, Liquid, Gas
- For example, A Person
 - In shopping mall behave like a customer
 - In bus behave like a passenger
 - In hospital behave like a patient
 - In home/house behave like a son/daughter
 - In university behave like a teacher /student.

Same person have **different behavior** in **different situations**.

- This is called Polymorphism.

Polymorphism

- It enables to write programs that process objects of classes (that are part of the same class hierarchy) as if they are all objects of the hierarchy's base class.

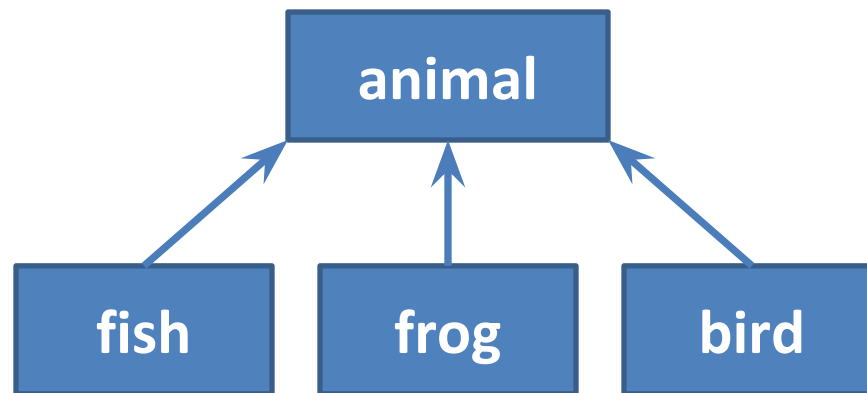


Polymorphism – Example 1

- Program that simulates the movement of several types of animals
 - Classes **Fish**, **Frog** and **Bird**
 - Each of these classes inherits from base class **Animal**
 - contains a function move and maintains an animal's current location
 - Each derived class implements function move
 - Program maintains an array of pointers to objects of the various derived classes (**Fish**, **Frog** and **Bird**)
 - To simulate the animal's movements, the program sends each object the same message, move()

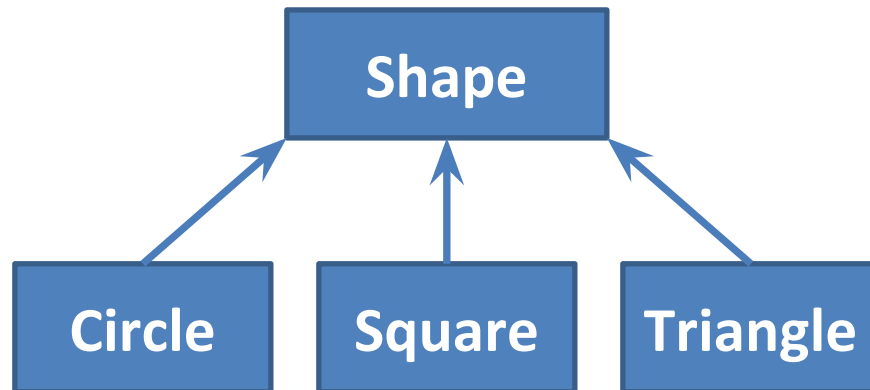
Cont.

- Each object knows how to modify its location appropriately for its specific type of movement
- Relying on each object to know how to "do the right thing" in response to the same function call is the key concept of polymorphism



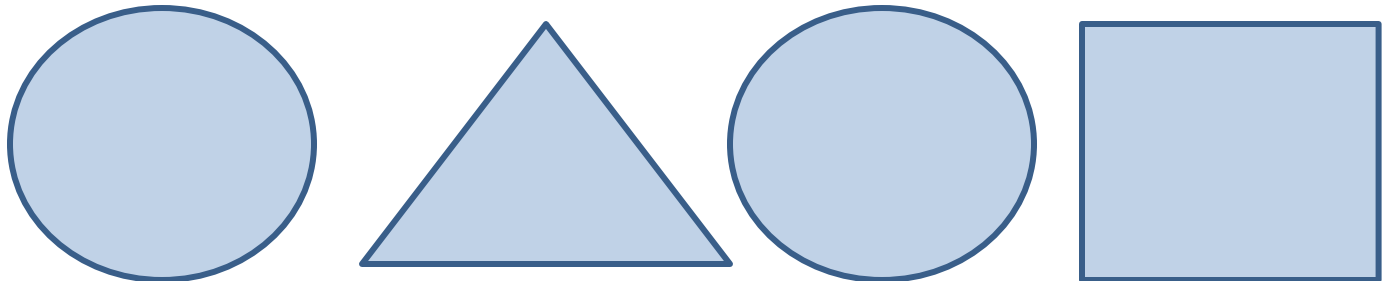
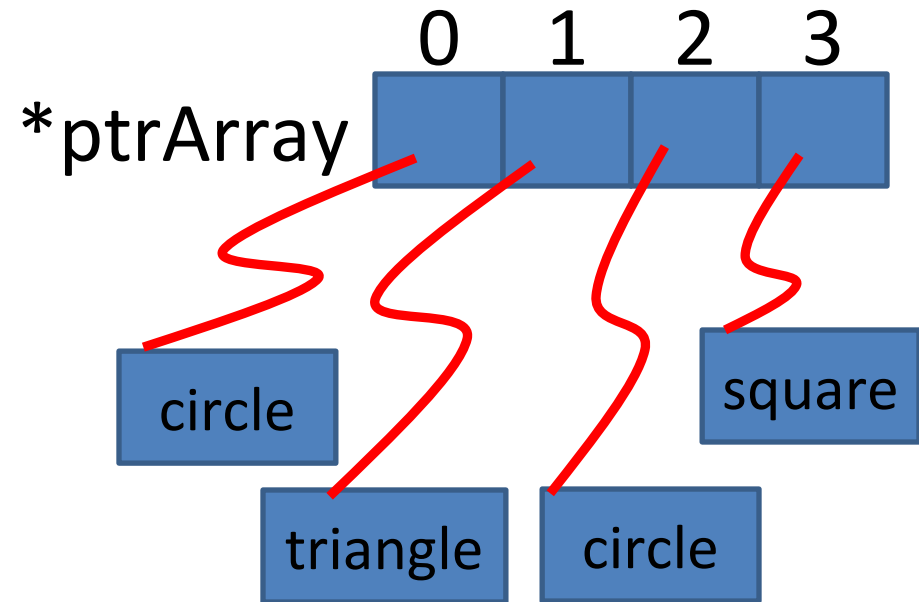
Polymorphism – Example 2

- Suppose you have a objects of different classes
 - All are inherited from a base class
 - you want to put them all in an array
 - perform a particular operation on them using the same function call.



Cont.

```
shape* ptrArray[4];  
ptrArray[0] = new Circle;  
ptrArray[1] = new Triangle;  
ptrArray[2] = new Circle;  
ptrArray[3] = new Square;  
for(int j=0; j<4; j++)  
    ptrarr[j]->draw();
```



Cont.

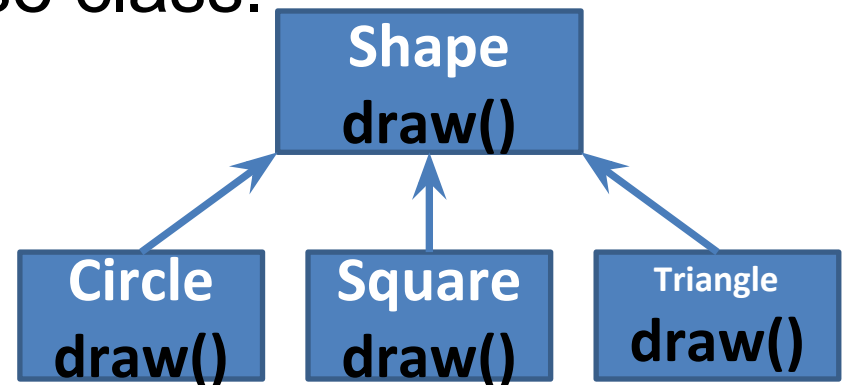
- This is an amazing capability:
 - Completely different functions are executed by the same function call.
 - If the pointer in ptrarray points to a circle, the function that draws a circle is called
 - if it points to a triangle, the triangle-drawing function is called.

This is called polymorphism, which means different forms

Condition for polymorphism

- Following condition must be met in order to achieve polymorphic behaviour
 - First, all the different classes of shapes, such as circle and triangles, must be inherited from a single base class
 - Second, the draw() function must be declared to be **virtual** in the base class.

ptrarr[j]->draw();



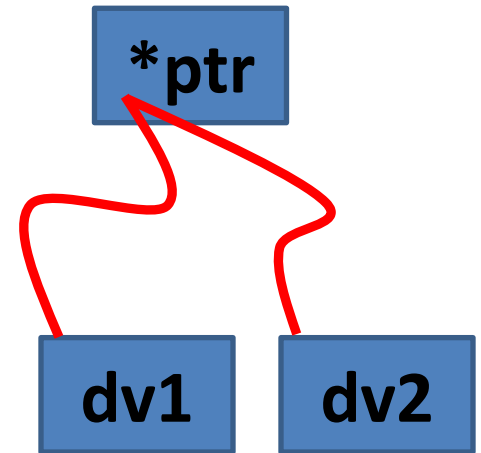
Example programs

- Normal Member Functions Accessed with Pointers
- Virtual Member Functions Accessed with Pointers

Normal Member Functions Accessed

```
class Base {  
    public:  
    void show()  
    { cout << "Base\n"; }  
};  
class Derv1 : public Base {  
    public:  
    void show()  
    { cout << "Derv1\n"; }  
};  
class Derv2 : public Base {  
    public:  
    void show()  
    { cout << "Derv2\n"; }  
};
```

```
main() {  
    Derv1 dv1;  
    Derv2 dv2;  
    Base* ptr;  
    ptr = &dv1;  
    ptr->show();  
    ptr = &dv2;  
    ptr->show();  
}
```



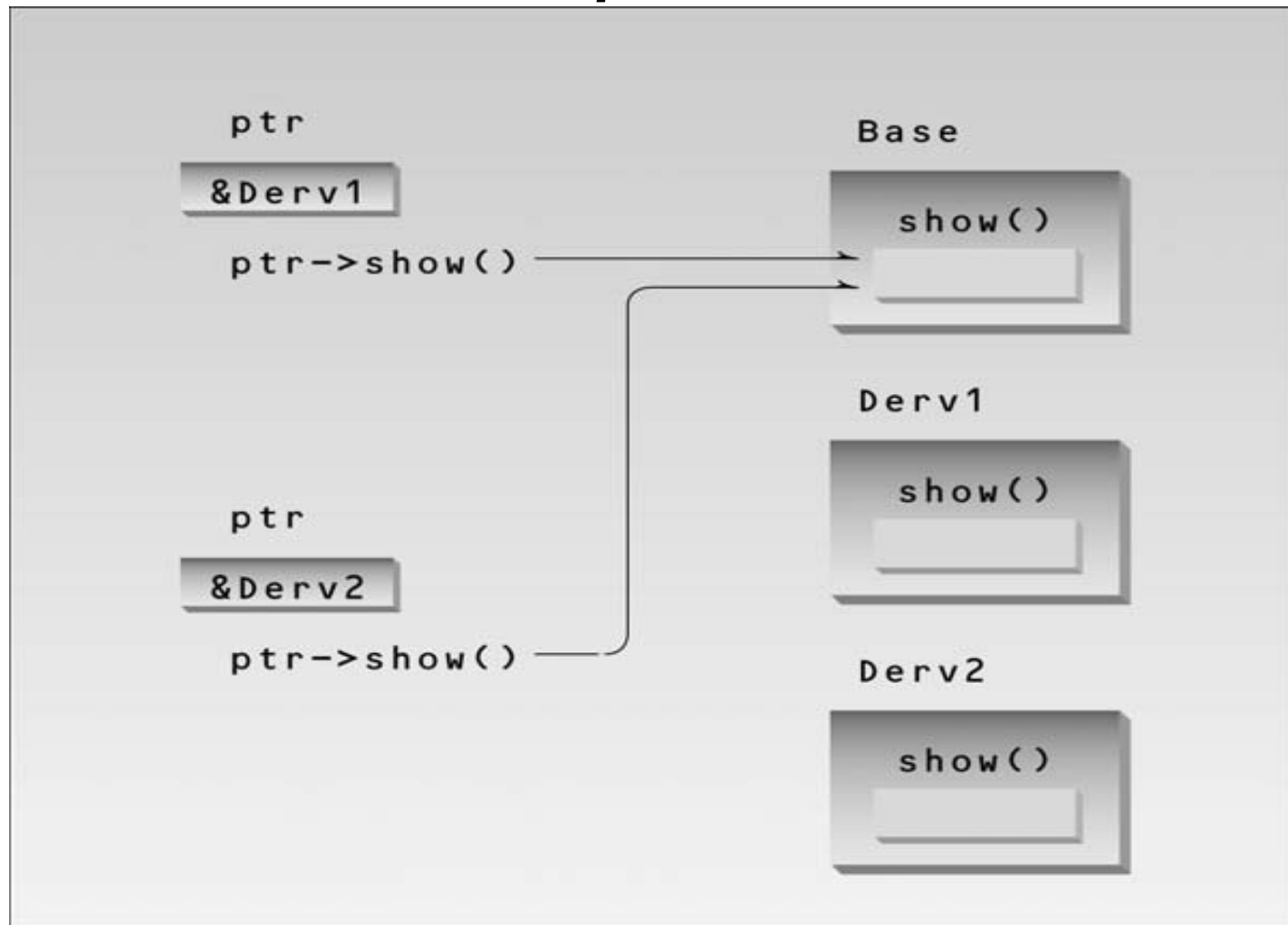
Program Output

Base

Base

Note: Compiler ignores the *contents of the pointer ptr* and chooses the member function that matches the type of the pointer

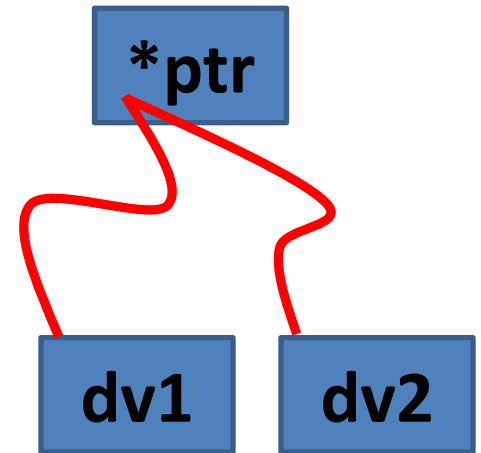
Non-virtual pointer access.



Virtual Member Functions Accessed

```
class Base {  
    public:  
    virtual void show()  
    { cout << "Base\n"; }  
};  
class Derv1 : public Base {  
    public:  
    void show()  
    { cout << "Derv1\n"; }  
};  
class Derv2 : public Base {  
    public:  
    void show()  
    { cout << "Derv2\n"; }  
};
```

```
main() {  
    Derv1 dv1;  
    Derv2 dv2;  
    Base* ptr;  
    ptr = &dv1;  
    ptr->show();  
    ptr = &dv2;  
    ptr->show();  
}
```



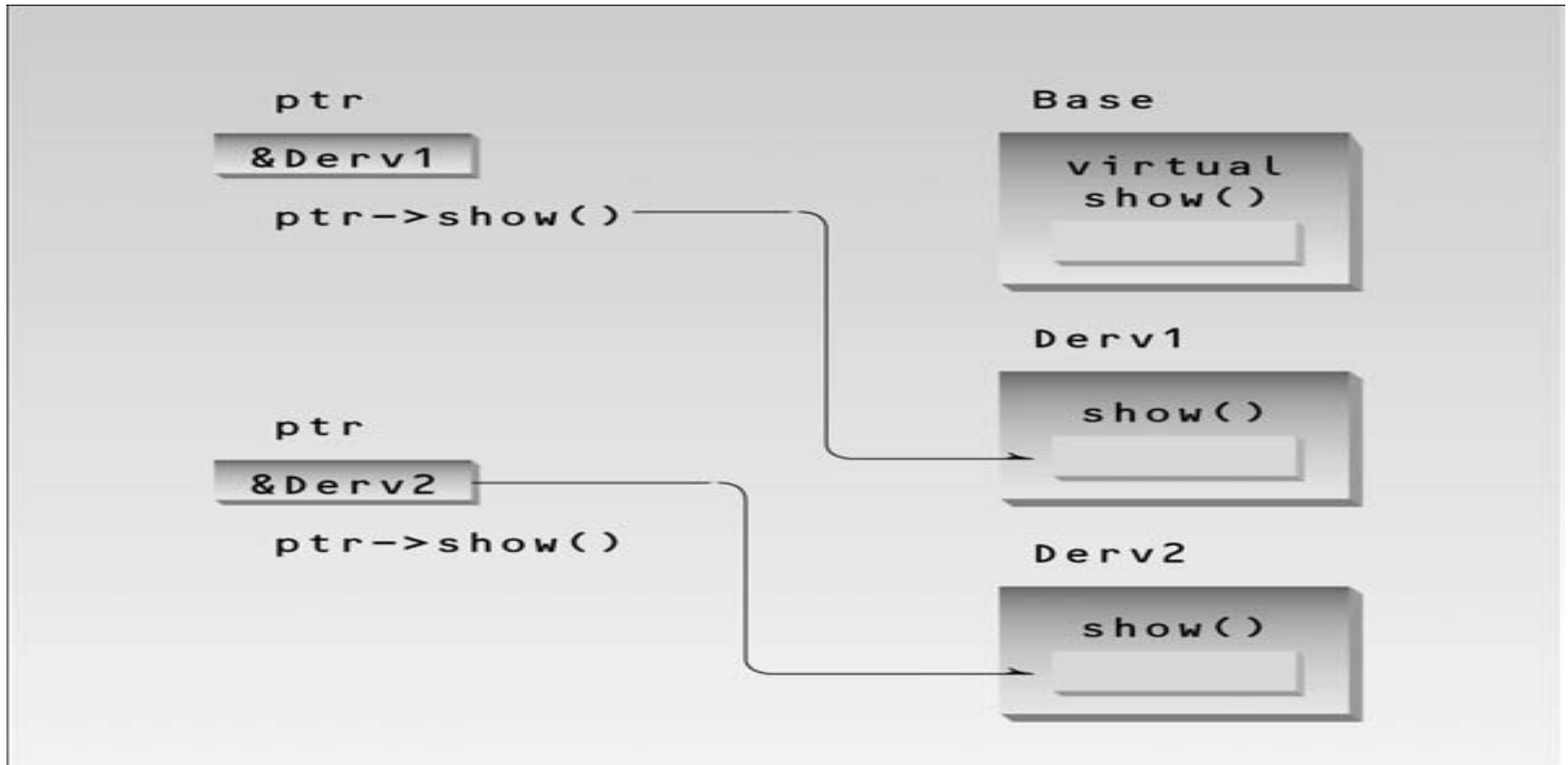
Program Output

Derv1

Derv2

Note: Compiler selects the function based on the *contents of the pointer ptr*, not on the *type of the pointer*

Virtual pointer access



Static binding/ Early binding/Compile time polymorphism

- When a member function is called with the object of that class
- Function invocation is resolved at compile time.
- This is called static or early binding
- This is not a polymorphic behavior
- Eg
- `der1.show()`
- `der2.show`

Dynamic binding/late binding/Run time polymorphism

- Virtual function can be invoked using base class pointer to a derived class object
 - `basePtr—>draw()`
- Now the correct derived class draw function will be selected dynamically (execution time)
- This is called dynamic or late binding
- At run time, when it knows what class is pointed by `basePtr`, the `show()` function of that class is called

Allowable assignments b/w base and derived class object and pointer

- Four ways to aim base and derive class pointer at base and derived class objects
 - Base class pointer to base class objects
 - Derived class pointer to derived class objects
 - Base class pointer to derive class objects
 - Derive class object is also object of base class
 - Derive class pointer to base class object.
 - Not allowed
 - “is a” relationship applies only from derive class to base class not vice versa

Abstract Classes and Pure Virtual Functions

```
class Base {  
    public:  
    virtual void show() = 0;  
};
```

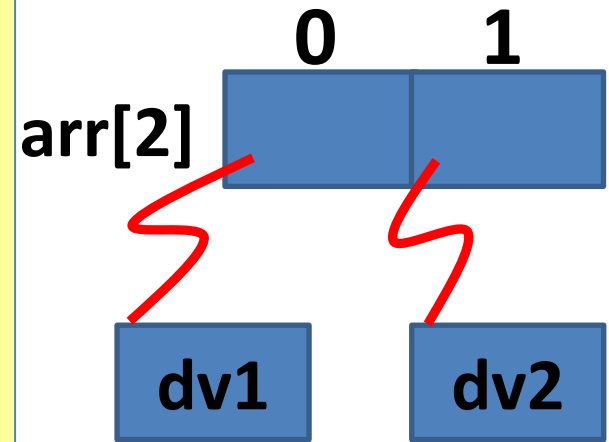
```
class Derv1 : public Base {  
    public:  
    void show()  
    { cout << "Derv1\n"; }  
};
```

```
class Derv2 : public Base {  
    public:  
    void show()  
    { cout << "Derv2\n"; }  
};
```

Pure virtual function

```
main() {  
    Base* arr[2];  
  
    Derv1 dv1;  
    Derv2 dv2;  
    arr[0] = &dv1;  
    arr[1] = &dv2;  
  
    arr[0]->show();  
    arr[1]->show();  
}
```

// Base b;



Program Output

Derv1

Derv2

Note: objects of abstract class cannot be created

Note

- After aiming a base-class pointer at a derived-class object, attempting to reference derived-class only members with the base-class pointer is a compilation error

```
class B {  
    public:  
    virtual void show() = 0;  
};
```

```
class Derv1 : public B {  
    public:  
    void show()  
    { cout << "Derv1\n"; }  
    void display()  
    {cout<<"hello derv1"; }  
};
```

```
main(){  
    B *ptr;  
    Derv1 dv1;  
    ptr = &dv1;  
    ptr->show();  
    ptr->display();  
}
```

Virtual Destructors

- A problem can occur when using polymorphism to process dynamically allocated objects of a class hierarchy
- E.g. applying delete operator to a base class pointer, which point to derived class objects
 - B *ptr;
 - Derv1 dv1;
 - ptr = &dv1;
 - ~~delete ptr;~~
- Solution : virtual destructor

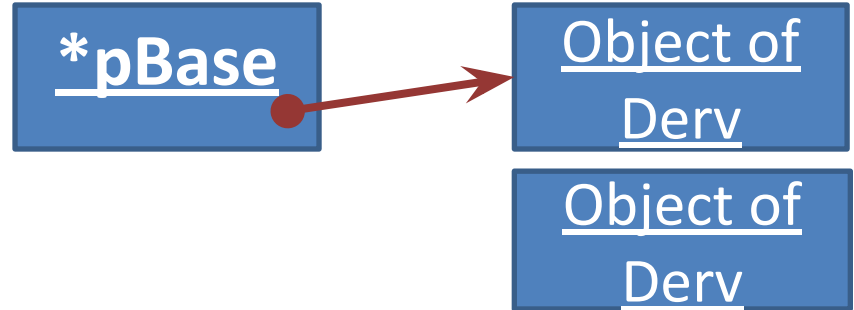
Cont.

- A virtual destructor in base class makes all derived class destructor virtual
 - B *ptr; Derv1 dv1;
 - ptr = &dv1;
 - delete ptr; ✓
- Now destructor for appropriate class is called
- When derived class object is destroyed, base class part of derive class object is also destroyed
- Base class destructor is executed after derived class destructor

Example – virtual destructor

```
class Base {  
    public:  
        virtual ~Base()  
        { cout << "Base  
        destroyed\n"; }  
};  
class Derv : public Base {  
    public:  
        ~Derv()  
        { cout << "Derv  
        destroyed\n"; }  
};  
main() {  
    Base* pBase = new Derv;  
    delete pBase;  
}
```

Without virtual Destructor



Program Output
Base destroyed

With virtual Destructor

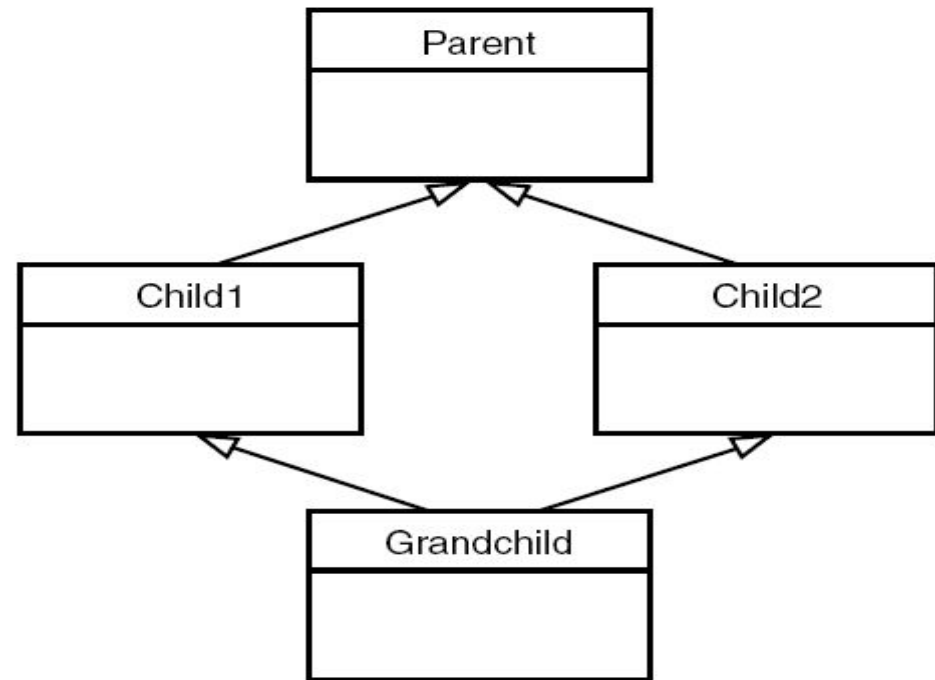


Program Output
Derv destroyed
Base destroyed

Virtual base classes

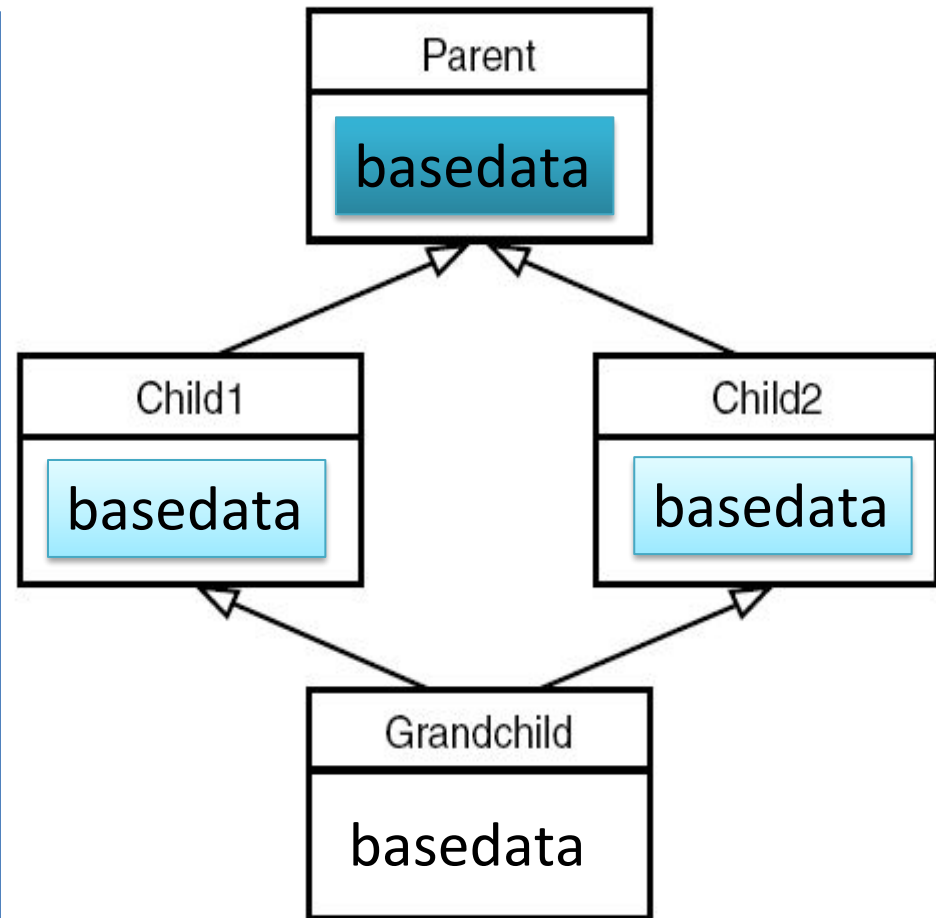
- A problem can arise if a member function in the *Grandchild* class wants to access data or functions in the *Parent* class

When the *Child1* and *Child2* classes are derived from *Parent*, each inherits a copy of *Parent*; this copy is called a subobject



Cont.

```
class Parent {  
    protected:  
        int basedata;  
};  
class Child1 : public Parent  
{ };  
class Child2 : public Parent  
{ };  
class Grandchild : public Child1,  
                    public Child2 {  
    public:  
        int getdata()  
        { return basedata; }  
};
```



This situation is ambiguous, and that's what the compiler reports

Cont.

```
class Parent {  
    protected:  
        int basedata;  
};  
class Child1 : virtual public Parent  
{ };  
class Child2 : virtual public Parent  
{ };  
class Grandchild : public Child1,  
public Child2 {  
    public:  
        int getdata()  
        { return basedata; }  
};
```

keyword virtual in these two classes causes them to share a single common Sub object of their base class Parent

Friends function

- Non member function should not be able to access the private data of that class.
- However there are situations to where these restriction lead to considerable
- Imagine a function to operate on object of two different classes.
 - `Void display(alpha a,beta b)`
 - `Display()` must be a friend function of alpha and beta class.

```
class Beta
class Alpha {
private:
    x:
public:
    Alpha(){
        x=3;
    }
    friend frifunc(alpha,beta);
};
```

```
class Beta {
private:
    y:
public:
    beta(){
        y=7;
    }
    friend frifunc(alpha,beta);
};
```

```
int frifunc(alpha a,beta b)
{
    return a.x+b.y.
};
```

Thanks
Any Question